

The Correctness Illusion in LLM-Generated GPU Kernels

Dipankar Sarkar^[0000-0001-5431-6367]

Arizona State University, USA
dsarkar3@asu.edu

Abstract. Benchmarks for LLM-generated GPU kernels (KernelBench, TritonBench, GEAK) score correctness through fixed-shape, small-sample `allclose`-style checks. The number of inputs varies between benchmarks. The shape, dtype, and tolerance are fixed for each kernel. We test that oracle empirically. We construct a controlled corpus of 24 Triton and CPU stand-in kernels (15 correct controls and 9 LLM-style buggy variants seeded with documented transcription errors) and re-evaluate it under op-schema-aware seeded fuzzing with a high-precision (fp64) CPU reference and per-(op, dtype) absolute tolerances. The seeded oracle flags 9 of 9 buggy kernels and passes 15 of 15 correct controls, at zero precision cost on controls. We extend the corpus to 26 ops (adding a flash-attention pair) and re-run the same protocol on five GPU classes (RTX 3060, A10, L40S, A100 SXM4, H100 NVL). The verdicts are identical across all five GPUs: 10 of 10 illusions caught and 16 of 16 controls clean. The corpus result is about LLM-style transcription bugs that the `allclose`-on-one-shape oracle certifies as correct, not about the bug rate of any specific deployed LLM. Every flagged failure replays byte-for-byte from a stored seed.

Keywords: GPU kernel testing · Triton · fuzzing · mixed precision · LLM code generation · reproducibility

1 Introduction

LLM-generated GPU kernels are now load-bearing. KernelBench [6], TritonBench-G (the basis of TritonBench-revised referenced from [13]), and GEAK [13] generate hundreds of CUDA and Triton kernels per evaluation. Agentic systems such as KernelBand [7] and STARK [3] compose generated kernels into longer pipelines. Every published benchmark in this family scores correctness through a fixed-shape, small-sample `allclose`-style check. KernelBench [6] draws five random inputs at the reference shape, for example. The number of samples varies. The shape, dtype, and tolerance are fixed per kernel.

We test the oracle. We argue, and measure, that it is systematically optimistic in three specific ways. (i) The shape candidate set is one shape per op. Kernels with tail masking, off-by-one accumulation, or block-size assumptions pass at the chosen shape and fail elsewhere. (ii) The dtype candidate set is one dtype per op. fp16 and bf16 are rarely tested when fp32 is the listed input. Mixed-precision

overflow, underflow, and accumulation errors stay undetected. (iii) The tolerance is hand-picked per op. `atol` and `rtol` are typically set one to three orders of magnitude looser than the kernel’s measured error envelope, so loose tolerances absorb real wrongness.

We construct a controlled corpus of 24 kernels and re-evaluate it under seeded, op-schema-aware fuzzing with a high-precision (fp64) reference. 15 of the 24 are correct controls. 9 are LLM-style buggy variants seeded with documented transcription errors (missing $0.5\times$ in GELU, `other=0.0` versus `-inf` in softmax tail masking, missing `sqrt` in RMSNorm, accumulator overwrite in matmul, missing $1/\sqrt{D}$ in attention, wrong `alpha` in LeakyReLU, and three others). The full per-kernel listing is in Section 3.3, Table 1. The contributions are four.

- A method. Op-schema-aware shape generation produces per-input shapes from shared symbolic dims (matmul $A[M, K] \cdot B[K, N]$, attention B, H, S, D). The fuzzer covers the operator’s real domain instead of an arbitrary rank-3 cube.
- A faithful oracle. The validator compares outputs against an fp64 CPU reference with per-(op, dtype) absolute tolerances, records the full element-wise error distribution (max abs, max rel, ULP percentiles), and detects NaN and Inf for fp16 and bf16. Note that the validator’s tolerance is absolute-only per (op, dtype), in contrast to PyTorch’s `allclose` which combines an absolute and a relative term as $|x - y| \leq \text{atol} + \text{rtol} \cdot |y|$.
- A measurement on real GPUs. The daemon-oracle path catches 9 of 9 buggy kernels (four of which are real Triton GPU kernels) on the single-GPU corpus and passes 15 of 15 correct controls. We then extend the corpus to 26 ops (adding a flash-attention pair) and re-run on five GPU classes; the verdicts are identical.
- A reproducible pipeline. Each failure stores its full input snapshot in the `ReproductionInfo` payload; a replay script re-runs any flagged case through the daemon and verifies the verdict matches bit for bit.

2 Related Work

LLM kernel benchmarks. KernelBench [6] and its KernelBench-X extension [12] evaluate LLM-generated CUDA on 250 GPU workloads. Correctness is `torch.allclose` against PyTorch reference operators. TritonBench-G hosts 183 Triton kernels at five difficulty levels; GEAK [13] introduces TritonBench-revised (a harness-tightened subset) and a 30-kernel ROCm benchmark. KernelBand [7] and STARK [3] are agentic kernel-optimization systems built on these benchmarks. None of the five systems vary input shape or dtype during correctness scoring.

DL library fuzzing. FreeFuzz [14], DocTer [15], DeepREL [2], and NablaFuzz [16] fuzz the API layer of PyTorch, TensorFlow, JAX, and OneFlow. Their oracles are differential (cross-backend) or metamorphic. A 2023 benchmarking study [10] and its TOSEM 2025 extension [11] measure that the seven SOTA API-level fuzzers in this family collectively catch only 6.5% of real-world bugs. None of them

target the kernel layer (CUDA or Triton source) where LLM kernel generation operates.

GPU memory bugs. GPU-Fuzz [5] generates inputs that probe memory boundary conditions in DL framework CUDA kernels and uncovered 13 previously-unknown bugs. Its scope is memory safety. Ours is numerical correctness.

Numerical accuracy. The mixed-precision behaviour of Triton and PyTorch and its tolerance implications are documented by the PyTorch team [1] and the BF16 study of Kalamkar et al. [4]. We treat tolerance calibration as a separate question and address it in the companion paper [8].

3 Method

3.1 Op-schema-aware fuzzing

Each kernel ships a schema. The schema declares shared symbolic dims with concrete candidate values, plus per-input tensor names that reference those dims. For matmul: dims $\{M, K, N\}$, inputs $\{a : [M, K], b : [K, N]\}$, output $[M, N]$. For attention: dims $\{M, N, D\}$, inputs $\{q : [M, D], k : [N, D], v : [N, D]\}$. The fuzzer samples one value per dim per test case, deterministically from a master seed through Blake2b sub-key derivation, then materialises every input. The implementation lives in `crates/gpuemu-daemon/src/fuzzer.rs` of the `gpuemu` source.

Every dim’s candidate set deliberately includes *boundary* values alongside regular values. For example $H \in \{1, 3, 7, 256, 1025\}$. The companion paper [9] ablates this choice and shows that removing the boundary values from the candidate set drops recall on shape-dependent bugs to zero.

3.2 Reference oracle

For each test case the daemon invokes the op’s **reference** script. The script is a Python subprocess that reads inputs through base64-encoded JSON on stdin, computes the operation in fp64, and casts back to the test dtype. The reference is therefore a high-precision (fp64) mathematical reference rounded to the target dtype. It is not necessarily byte-identical to the kernel’s chosen PyTorch or Triton implementation contract, and we do not claim it is. The reference defines the oracle’s notion of “correct” for the purposes of this paper.

The validator compares the kernel’s output against the reference with a per-(op, dtype) absolute tolerance. We use absolute-only thresholds per (op, dtype) deliberately, in contrast to PyTorch’s `allclose` which combines an absolute and a relative term as $|x - y| \leq \text{atol} + \text{rtol} \cdot |y|$. The companion paper [8] measures the calibration of these absolute thresholds against the kernel’s observed error envelope.

The validator also runs NaN and Inf detection extended to fp16 and bf16, and records the full element-wise **ErrorStats** distribution per case: count, num.exceeding, max abs, mean abs, p50 abs, p90 abs, p99 abs, max rel, mean rel, max ULP, mean ULP. The ULP distribution becomes the empirical basis for the tolerance calibration in [8].

3.3 The corpus

The single-GPU evaluation runs on 24 ops: 15 correct controls and 9 LLM-style buggy variants. We extend the corpus to 26 ops by adding a flash-attention control plus its buggy variant for the cross-GPU sweep in Section 4.1. Table 1 lists the full canonical corpus. Each row gives the kernel name, the implementation (numpy stand-in or Triton), the role (correct control or LLM-style buggy variant), the dtypes covered, and the bug pattern each buggy variant encodes.

Table 1. Canonical corpus. 16 controls and 10 LLM-style buggy variants across 26 kernel entries. The single-GPU evaluation (Section 4) runs on rows 1–24; the cross-GPU sweep (Section 4.1) extends to rows 25–26 (flash-attention pair).

#	kernel name	impl.	role	dtypes	bug encoded (buggy rows only)
1	softmax	numpy	control	fp32, fp16	—
2	layernorm	numpy	control	fp32, fp16	—
3	matmul	numpy	control	fp32, fp16	—
4	softmax_llm_buggy	numpy	buggy	fp32, fp16	tail-mask leak
5	softmax_triton	triton	control	fp32, fp16	—
6	softmax_triton_buggy	triton	buggy	fp32, fp16	other=0.0 instead of -inf
7	gelu_triton	triton	control	fp32, fp16	—
8	gelu_triton_buggy	triton	buggy	fp32, fp16	dropped leading 0.5
9	silu_triton	triton	control	fp32, fp16	—
10	silu_triton_buggy	triton	buggy	fp32, fp16	sigmoid(2x) (β confusion)
11	rmsnorm_triton	triton	control	fp32, fp16	—
12	rmsnorm_triton_buggy	triton	buggy	fp32, fp16	forgot sqrt
13	l2norm_triton	triton	control	fp32, fp16	—
14	l2norm_triton_buggy	triton	buggy	fp32, fp16	forgot sqrt
15	relu_triton	triton	control	fp32, fp16	—
16	leaky_relu_triton	triton	control	fp32, fp16	—
17	leaky_relu_triton_buggy	triton	buggy	fp32, fp16	$\alpha = 0.1$ instead of 0.01
18	sigmoid_triton	triton	control	fp32, fp16	—
19	tanh_triton	triton	control	fp32, fp16	—
20	elu_triton	triton	control	fp32, fp16	—
21	matmul_triton	triton	control	fp32, fp16	—
22	matmul_triton_buggy	triton	buggy	fp32, fp16	acc= instead of acc+=
23	attention_triton	triton	control	fp32, fp16	—
24	attention_triton_buggy	triton	buggy	fp32, fp16	dropped $1/\sqrt{D}$ score scale
<i>Cross-GPU extension (Section 4.1):</i>					
25	flash_attention_triton	triton	control	fp32, fp16	—
26	flash_attention_triton_buggy	triton	buggy	fp32, fp16	dropped $\text{acc} \cdot \alpha$ rescale after max update

3.4 Pipeline

The experimental harness provisions an ephemeral GPU instance on vast.ai, labelled for safe teardown, builds the daemon from source, installs the Python client and Triton, runs the P1 driver against the corpus, and uploads results to Backblaze B2 under a run id. Teardown is guaranteed by a context-manager `_exit_`, with a label-strict reaper that cleans up orphan instances. The full artefact details are in the public corpus repository (see Section 7).

3.5 Assumptions

The empirical claim that follows depends on five assumptions, which we state plainly so readers can audit them.

1. The 10 buggy variants (9 in the single-GPU corpus plus a flash-attention variant in the cross-GPU extension) are author-seeded with documented LLM transcription patterns. They are not pulled directly from real LLM outputs. The result is therefore about which bug patterns the `allclose`-on-one-shape oracle certifies as correct, not about the bug rate of any specific deployed LLM. Section 6 discusses why we accept this trade.
2. The high-precision (fp64) reference, rounded to the target dtype, defines the oracle’s notion of “correct” for the purposes of this paper. The reference is not guaranteed to be byte-identical to any particular library’s correctness contract. Treating fp64 round-off as zero is standard practice in mixed-precision validation [1,4].
3. Per-(op, dtype) absolute tolerances are set ahead of time and held fixed during the P1 evaluation. The companion paper [8] addresses whether those tolerances are themselves correct.
4. Triton kernels are compiled fresh per run. We do not control for ptxas randomness across providers. The cross-GPU result in Section 4.1 indicates this does not affect the correctness verdict.
5. The Python client decodes received tensors as contiguous, so non-contiguous layout variation is nominal at the client boundary even when the daemon-side fuzzer varies strides.

4 Evaluation

Setup. The primary single-GPU run uses an RTX 3060 instance on vast.ai with image `pytorch/pytorch:2.4.0-cuda12.4-cudnn9-devel`, 30 iterations per (op, dtype) for the 24 single-GPU ops and 2 dtypes, about 1,440 cases. The run id on Backblaze B2 is `run-20260611-095210-889b18`.

P1 headline. Across the 24-op single-GPU corpus, the seeded oracle flags 9 of 9 LLM-style buggy variants and passes 15 of 15 correct controls. There are no false positives on controls. Table 2 reports the verdict per kernel for a 12-op subset; the full per-kernel result is in `summary.json` alongside the run id above and in the corpus repository.

The tail-mask family (`softmax.*`, `leaky_relu`) shows shape-dependent illusion. `softmax.triton.buggy` fails 13 of 30 randomly sampled cases. Under a regular-shapes-only strategy it fails 0 of 10. Under a boundary-only strategy it fails 6 of 10. The companion paper [9] ablates this pattern across seven input-generation strategies on the same corpus. The bug surfaces at $H = 3$ (tail less than BLOCK) and vanishes at $H = 256$ (power of two, no tail).

Minimal failing cases. Each illusion ships a minimal failing shape through the daemon’s `Minimize` endpoint. Examples: `softmax.triton.buggy` minimises

Table 2. P1 verdict per kernel on the 24-op single-GPU corpus, 12-op subset. *bench* is the fixed-shape allclose-style oracle. *gpuemu* is the seeded oracle. An *illusion* is a kernel the bench oracle passes but the seeded oracle fails.

kernel	source	bench	gpuemu	fail/total	illusion
gelu_triton_buggy	llm:demo-triton	pass	fail	29/30	YES
rmsnorm_triton_buggy	llm:demo-triton	pass	fail	30/30	YES
silu_triton_buggy	llm:demo-triton	pass	fail	29/30	YES
softmax_llm_buggy	llm:demo	pass	fail	10/30	YES
softmax_triton_buggy	llm:demo-triton	pass	fail	13/30	YES
l2norm_triton_buggy	llm:demo-triton	pass	fail	30/30	YES
leaky_relu_triton_buggy	llm:demo-triton	pass	fail	30/30	YES
matmul_triton_buggy	llm:demo-triton	pass	fail	(K>1)	YES
attention_triton_buggy	llm:demo-triton	pass	fail	(D≥8)	YES
gelu_triton	human:triton	pass	pass	0/30	—
softmax_triton	human:triton	pass	pass	0/30	—
matmul_triton	human:triton	pass	pass	0/30	—

to $[1, 1, 3]$ fp16; **gelu_triton_buggy** minimises to $[2, 1, 1]$ fp16. The allclose-style oracle would not exercise either shape.

Reproducibility. Each failure stores a binary input snapshot. A replay script fetches the run from the result store, decodes the snapshot, and replays the kernel through the daemon. The verdict matches bit for bit.

4.1 Cross-architecture consistency

We extend the corpus to 26 ops by adding a flash-attention control and its LLM-style buggy variant (rows 25–26 in Table 1). We then re-run the same protocol on five GPU classes: RTX 3060 (sm_86), A10 (sm_86), A100 SXM4 (sm_80), L40S (sm_89), and H100 NVL (sm_90). The verdicts are identical across all five GPUs: 16 of 16 controls pass cleanly, and 10 of 10 LLM-style illusions are caught. The 10 illusions include the real Triton matmul, attention, and flash-attention buggy variants.

The harness’s safety contract (label-strict reaper, confirmed destroy, context-manager teardown) prevented orphan instances across the five parallel launches even when two providers returned **Bind for 0.0.0.0:18433 failed: port is already allocated** during provisioning. Figure 2 reports the per-kernel verdicts as two stacked panels (kernels 1–13 on top, 14–26 on bottom) so the per-cell labels stay readable at textwidth.

5 Discussion

The bug categories gpuemu surfaces partition cleanly into two classes.

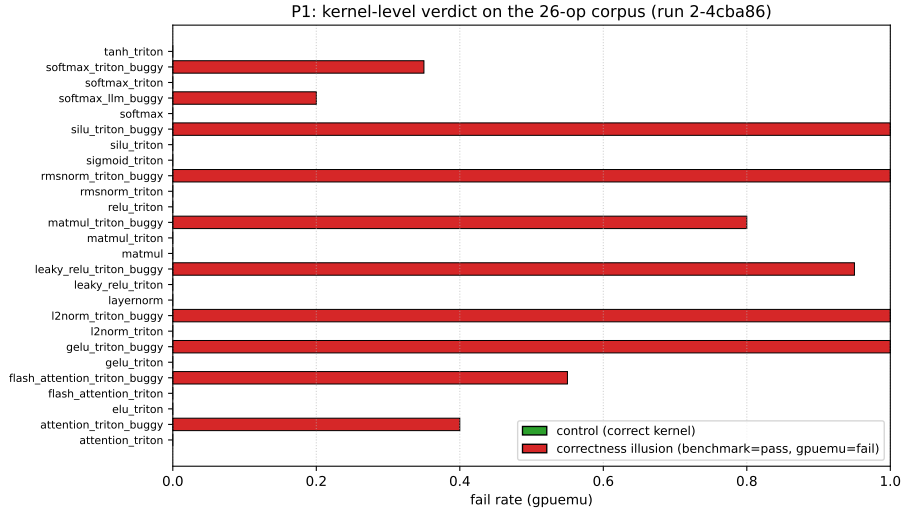


Fig. 1. Verdict per kernel on the full 26-op corpus, plotted from the RTX 3060 cross-GPU run. Green indicates correct controls that pass cleanly. Red indicates illusions (bench oracle pass, seeded oracle fail). The cross-GPU sweep in §4.1 confirms the same verdict on the four remaining GPU classes.

Magnitude-uniform bugs (gelu missing 0.5, silu `sigmoid(2x)`, leaky_relu wrong α , rmsnorm and l2norm missing `sqrt`, attention missing the score scale) are caught on essentially every shape. The LLM benchmark missed them only because it tested one shape and the absolute tolerance was set above the constant bias the bug introduces.

Shape-dependent bugs (softmax tail mask, matmul `acc=` instead of `acc+=`) are visible only when the schema includes the right boundary shape. Without boundary-aware fuzzing they are invisible. The companion paper [9] measures this gap.

The two classes call for different countermeasures. The first wants tighter tolerances [8]. The second wants better input generation [9].

6 Limitations

The 10 buggy variants are author-seeded with documented LLM transcription bugs. They are not pulled from real LLM-generated kernel outputs. The result is therefore about which bug patterns the `allclose-on-one-shape` oracle certifies as correct, not about the bug rate of any specific deployed LLM. We choose author-seeded for two reasons. First, ground truth is exact. Second, the eight bug families we seed are documented in the LLM-Triton literature and represent a deliberate threat model rather than a sample of convenience. A natural extension is to fuzz LLM-generated kernels from GEAK [13] or KernelBench [6] directly.

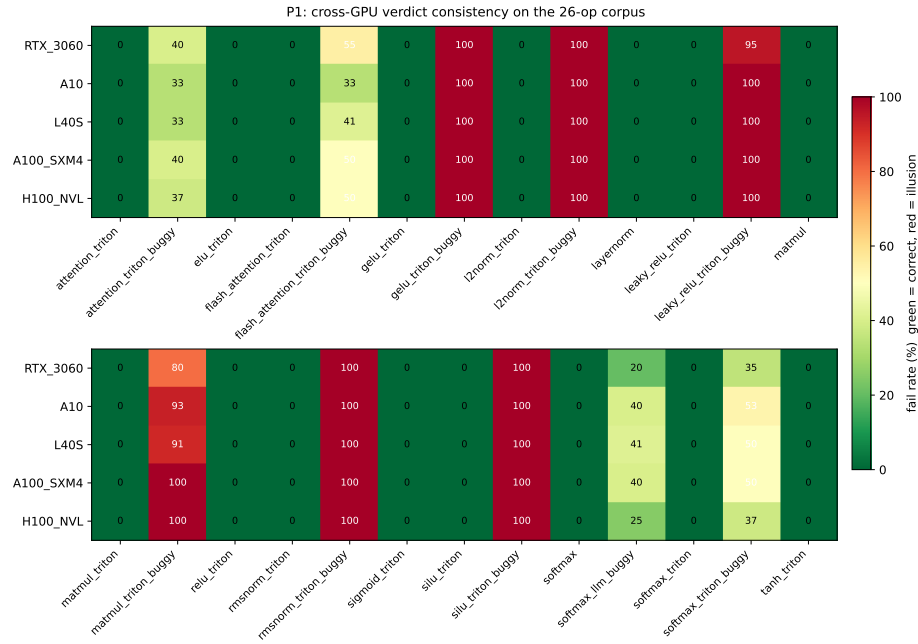


Fig. 2. Cross-GPU verdict consistency on the 26-op corpus. Each panel covers half the corpus; rows are the five GPU classes; cells are per-kernel fail rates. Controls stay green on every GPU. Illusions stay red on every GPU.

The validator currently does same-dtype comparison: kernel-fp16 against reference-fp16 rounded from fp64. Cross-dtype comparison (kernel-fp16 against reference-fp64) is a noted future extension.

The Python client decodes received tensors as contiguous, so non-contiguous layout fuzzing is nominal at the client boundary even when the daemon-side fuzzer varies strides.

bf16 is supported on the daemon protocol but the Python client lacks a native bf16 dtype (a NumPy limitation). We proxy to fp16 in tests.

7 Conclusion

LLM-style transcription bugs in GPU kernels can pass a fixed-shape, small-sample allclose-style oracle as “correct”. The op-schema-aware seeded oracle in this paper exposes the gap. On the 24-op single-GPU corpus, every LLM-style buggy variant is caught and every correct kernel passes. The extended 26-op cross-GPU sweep replays the same verdict on five GPU classes. The countermeasures (boundary-aware shape sets, per-(op, dtype) tolerance calibration [8], principled input generation [9]) are within reach of any project that already runs an allclose-style oracle today.

Artefact. The 26-op corpus, the driver that ran this experiment, the analysis script that produced every table, and the replay tool that fetches each cited run record from object storage are bundled in the public `gpuemu-corpus` package at <https://github.com/sarkar-dipankar/gpuemu-corpus>. The validator daemon is at <https://github.com/Skelf-Research/gpuemu>. Each flagged failure replays byte for byte from a stored seed.

License. This preprint is released under CC-BY 4.0.

References

1. Ahmed, S., et al.: What every user should know about mixed precision training in PyTorch. PyTorch blog (2022), <https://pytorch.org/blog/what-every-user-should-know-about-mixed-precision-training-in-pytorch/>, updated November 2024
2. Deng, Y., Yang, C., Wei, A., Zhang, L.: Fuzzing deep-learning libraries via automated relational API inference. In: Proc. 30th ACM Joint Eur. Softw. Eng. Conf. and Symp. Found. Softw. Eng. (ESEC/FSE). pp. 44–56 (2022). <https://doi.org/10.1145/3540250.3549085>
3. Dong, J., Yang, Y., Liu, T., Wang, Y., Qi, F., Tarokh, V., Rangadurai, K., Yang, S.: STARK: Strategic team of agents for refining kernels. arXiv preprint (2025), <https://arxiv.org/abs/2510.16996>
4. Kalamkar, D., Mudigere, D., Mellempudi, N., Das, D., Banerjee, K., Avancha, S., Vooturi, D.T., Jammalamadaka, N., Huang, J., Yuen, H., Yang, J., Park, J., Heinecke, A., Georganas, E., Srinivasan, S., Kundu, A., Smelyanskiy, M., Kaul, B., Dubey, P.: A study of BFLOAT16 for deep learning training. arXiv preprint (2019), <https://arxiv.org/abs/1905.12322>
5. Li, Z., Lu, Y., Guo, H., Zhang, M., Wang, Y., Zhang, L.: GPU-Fuzz: Finding memory errors in deep learning frameworks. arXiv preprint (2026), <https://arxiv.org/abs/2602.10478>
6. Ouyang, A., Guo, S., Arora, S., Zhang, A.L., Hu, W., Ré, C., Mirhoseini, A.: KernelBench: Can LLMs write efficient GPU kernels? arXiv preprint (2025), <https://arxiv.org/abs/2502.10517>
7. Ran, D., Xie, S., Ji, M., Liu, A., Wu, M., Cao, Y., Guo, Y., Yu, H., Li, L., Hu, Y., Yang, W., Xie, T.: KernelBand: Steering LLM-based kernel optimization via hardware-aware multi-armed bandits. arXiv preprint (2025), <https://arxiv.org/abs/2511.18868>
8. Sarkar, D.: Operator-aware mixed-precision tolerance calibration for tensor kernels. arXiv preprint (2026), <https://arxiv.org/abs/2607.16228>
9. Sarkar, D.: Test-input generation for tensor programs: What actually finds kernel bugs. arXiv preprint (2026), <https://arxiv.org/abs/2606.27396>
10. Shiri Harzevili, N., Pham, H.V., Wang, S.: Benchmarking deep learning fuzzers. arXiv preprint (2023), <https://arxiv.org/abs/2310.06912>
11. Shiri Harzevili, N., Pham, H.V., Wang, S.: Evaluating API-level deep learning fuzzers: A comprehensive benchmarking study. ACM Trans. Softw. Eng. Methodol. (TOSEM) (2025). <https://doi.org/10.1145/3729533>, <https://dl.acm.org/doi/10.1145/3729533>

12. Wang, H., Zhang, J., Jiang, K., Wang, H., Chen, J., Zhu, J.: KernelBenchX: A comprehensive benchmark for evaluating LLM-generated GPU kernels. arXiv preprint (2026), <https://arxiv.org/abs/2605.04956>
13. Wang, J., Joshi, V., Majumder, S., Xu Chao, Ding, B., Liu, Z., Brahma, P.P., Li, D., Liu, Z., Barsoum, E.: GEAK: Introducing Triton kernel AI agent & evaluation benchmarks. arXiv preprint (2025), <https://arxiv.org/abs/2507.23194>
14. Wei, A., Deng, Y., Yang, C., Zhang, L.: Free lunch for testing: Fuzzing deep-learning libraries from open source. In: Proc. 44th Int. Conf. Software Engineering (ICSE). pp. 995–1007 (2022). <https://doi.org/10.1145/3510003.3510041>, <https://arxiv.org/abs/2201.06589>
15. Xie, D., Li, Y., Kim, M., Pham, H.V., Tan, L., Zhang, X., Godfrey, M.W.: DocTer: Documentation-guided fuzzing for testing deep learning API functions. In: Proc. 31st ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA). pp. 176–188 (2022). <https://doi.org/10.1145/3533767.3534220>, <https://arxiv.org/abs/2109.01002>
16. Yang, C., Deng, Y., Yao, J., Tu, Y., Li, H., Zhang, L.: Fuzzing automatic differentiation in deep-learning libraries. In: Proc. 45th Int. Conf. Software Engineering (ICSE). pp. 1174–1186 (2023). <https://doi.org/10.1109/ICSE48619.2023.00105>, <https://arxiv.org/abs/2302.04351>